From Bounded Model Checking to Mechanized Proof: A Multi-Level Verification Case Study for a Price-Time Priority Matching Engine*

Ara Aslyan
Strike Technologies, New York
aaslyan@striketechnologies.com

04/11/2026

Abstract

We present a case study in multi-level formal verification of a feature-rich price-time priority matching engine—the software at the heart of every stock, futures, and cryptocurrency exchange. The engine supports five order types (LIMIT, MARKET, MARKET-TO-LIMIT, STOP_LIMIT, STOP_MARKET), iceberg display quantities, four self-trade prevention policies, post-only, fill-or-kill, and minimum-quantity semantics.

The paper is structured around the assurance layers and the defects they made visible. Bounded model checking with TLC, across configurations totaling 36 million reachable states, produced a concrete FIFO counterexample: triggered stop orders retained their original submission timestamp. The same modeling pass exposed a prose-specification gap in the interaction between STP DECREMENT and iceberg reload semantics. Mechanized proof found two unbounded/transcription issues: the matching loop's original constant fuel bound of 100 steps made the key preservation theorem false for books with more than 100 contra-side orders, and the full-invariant extension exposed an MTL/minimum-quantity routing error in the Lean and TLA+ transcriptions. Finally, replaying TLC-generated traces against a C++ implementation found a dispose defect: a triggered stop-market order with a persistent time-in-force could leave a market remainder resting on the book.

We prove in Lean 4 that the processing pipeline preserves the full book-state invariant suite of the specification's §13—not only that the book stays uncrossed and sorted on both sides (`AllInv`), but also no ghost orders, status consistency, FIFO within each price level, and no resting market, market-to-limit, or minimum-quantity orders, together with the post-only and self-trade-prevention guarantees on emitted trades—for arbitrary book sizes, with a fuel bound computed from the book state and proved sufficient. The proof spans three files totaling roughly 5,700 lines: a 3,965-line constructive core, a 1,423-line extension lifting the remaining §13 invariants, and a complementary 311-line structural proof via three-case analysis of the matching termination condition.

The overarching lesson is that feature composition is where bugs hide. Each individual feature is correctly specified in isolation. The bugs live in intersections that prose review cannot see but an operational model exposes by construction.

1 What Is a Matching Engine?

1.1 The Role of a Matching Engine

When you buy a stock, someone else is selling it. The matching engine finds the seller. Every exchange—stock market, futures exchange, cryptocurrency platform—runs a matching engine

*Artifacts available at https://github.com/formally-verified-exchange/verified-matching-engine.

at its core. Every second, it processes thousands of incoming orders and decides which ones trade with which, at what price, in what quantity, and in what sequence.

If the matching engine makes a mistake, real money moves wrong. An order that should trade does not. An order that should not trade does. An order sitting at the front of a queue is pushed behind a newer one. These are not theoretical concerns: every such error is a financial event with an identifiable winner and loser.

This paper is about formally verifying that a matching engine does what it claims to do—not just for small test cases, but for all possible inputs of all sizes.

1.2 Orders

An order is an instruction to trade. The simplest example: "I want to buy 100 shares at $150 or less." This is a BUY LIMIT order. The engine will fill it if a willing seller exists at that price or better; otherwise, it waits on the book.

The engine studied in this paper supports five order types:

- **limit**: rest at a specified price; fill at that price or better. "Buy 100 at $150." Will wait indefinitely for a matching seller.
- **market**: fill at whatever price is available right now; never rest on the book. "Buy 100 at any price." Executes immediately or not at all.
- **market-to-limit (MTL)**: aggressive first, then patient. Match at any available price; lock to the first execution price; rest any unfilled remainder as a limit order at that price.
- **stop_limit**: dormant until a trigger event, then active. "Once the last trade price reaches $155, enter a buy limit order at $156." Used for stop-loss and momentum strategies.
- **stop_market**: same trigger logic, but converts to a market order on activation rather than a limit order.

Orders also carry *modifiers* that change their behavior:

- **Iceberg** (display quantity): only a small "visible slice" is shown to the market. When that slice fills, the order reloads a fresh slice from its hidden reserve—but loses its place in the queue and joins the back of the line at its price.
- **Post-only**: the order must not aggress against the opposite side. If it would immediately trade, it is rejected. Useful for market makers who want to add liquidity, not consume it.
- **Fill-or-Kill (FOK)**: fill the entire order immediately, or cancel it entirely. No partial fills, no resting.
- **Minimum quantity (MinQty)**: the first fill must be at least this large; after the initial threshold is met, the remainder may rest and fill in smaller pieces.
- **Self-trade prevention (STP)**: four policies govern what happens when an incoming order would trade against a resting order from the same participant. Options: cancel the incoming order (CANCEL_NEWEST), cancel the resting order (CANCEL_OLDEST), cancel both (CANCEL_BOTH), or reduce both quantities by the overlapping amount (DECREMENT).

1.3 The Order Book

The order book is the matching engine's central data structure. It has two sides: bids (buy orders) and asks (sell orders). Each side is organized by price, with the most competitive prices at the top.

```
        BIDS (buyers)           ASKS (sellers)
  Price    Qty   Orders    |  Price    Qty   Orders
  ------  -----  -------   |  ------  -----  -------
  $151     200   [B, C]    |  $152     100   [D]
  $150     500   [A]       |  $153     300   [E, F]
  $149     150   [G]       |  $155     200   [H]
```

```
Best bid: $151          Best ask: $152
Spread: $1
```

In this book, the best bid is $151 and the best ask is $152. The *spread* is $1. No immediate trade is possible because no buyer is willing to pay as much as the cheapest seller is asking.

When a new buy order arrives at $152 or higher, it *crosses the spread*: it is willing to pay the ask price, so a trade occurs. The matching engine fills the incoming order against the cheapest resting ask, consuming quantity from the front of that level's queue, and repeating until the incoming order is satisfied or no crossable orders remain.

Within each price level, orders are served in *first-in, first-out* (FIFO) order: the order that arrived earliest has highest priority. This is the "time priority" in price-time priority. It means that among all sellers willing to sell at $152, the one who submitted first gets filled first.

1.4 What Must Be True—Always

A matching engine makes a set of guarantees. These must hold after every single operation, no exceptions:

1. **The book is not crossed.** The best bid is always strictly less than the best ask (or one side is empty). If the bid were higher than the ask, a trade should have occurred.
2. **The book is sorted.** Bids are arranged in strictly descending price order; asks in strictly ascending order. A buyer willing to pay more always has higher priority than one willing to pay less.
3. **FIFO within each price level.** Among orders at the same price, earlier arrivals are served first. Timestamps are strictly increasing down each level's queue.
4. **No ghost orders.** Every resting order has a positive remaining quantity. An order with zero quantity left must have been removed.
5. **Post-only safety.** A post-only order never results in a trade. If it would have aggressed, it was rejected before any trade occurred.
6. **Self-trade prevention.** No trade occurs between two orders belonging to the same STP group.

These rules sound obvious. They are obvious—individually. The difficulty is maintaining all of them simultaneously when multiple features interact. Guaranteeing rule 3 (FIFO) requires careful handling of every path by which an order can enter the book. Guaranteeing rule 6 (STP) requires careful handling of every order type, including stop orders that re-enter the pipeline from a dormant state. Features that interact with each other create combinatorial cases that no single human reviewer is likely to hold in mind simultaneously.

1.5 Where Bugs Hide: Feature Interactions

Each feature in a matching engine can be specified correctly in isolation. The danger is interactions. Consider:

- Stop orders are correctly described: they are dormant until triggered, then converted and processed.
- FIFO insertion is correctly described: orders are appended to the back of their price level's queue, with a timestamp reflecting their arrival time.

These two descriptions are each locally correct. But they interact: when a stop order is triggered, it is converted and re-enters the pipeline. If its timestamp is not updated at trigger time, it carries its original submission timestamp—which may be earlier than orders already resting on the book. It will be appended to the back of the queue, but its timestamp will be out of order. FIFO is violated.

A human reviewer reading the stop section and the FIFO section separately will not see this. Both sections are individually correct. The interaction is invisible unless you hold both simultaneously.

A second example: iceberg reload is correctly described for the fill path. But STP DECREMENT reduces a resting order's visible quantity without generating a trade—so it is not the fill path. If DECREMENT drives visible quantity to zero while hidden quantity remains, the iceberg never reloads, and the order is permanently stranded: it sits on the book, cannot match, and cannot reload.

This is the problem we set out to solve. The goal is a verification approach that does not read features section by section, but evaluates all of them simultaneously. The rest of this paper describes what we built and what we found.

2 How We Checked It—Overview

We verified the engine at two complementary levels, each with a different scope and a different kind of guarantee.

2.1 Level 1: Try Every Combination (Bounded Model Checking)

The first approach is exhaustive state-space exploration. We encoded the engine's rules as a mathematical model in TLA+ and used the TLC model checker to enumerate *every possible sequence of events* within a set of small configurations: up to 2 orders, quantities up to 2, prices drawn from a set of 2 or 3 values.

TLC is a brute-force tool. It generates every reachable state of the model and checks that each one satisfies every stated invariant. If any invariant is ever violated, TLC returns a concrete counterexample trace showing exactly which sequence of orders produced the violation.

Within the configurations we explored, TLC generated 36 million distinct states. **This produced one concrete invariant counterexample and exposed one additional specification gap.**

The limitation of this approach is its scope. Production matching engines hold millions of orders at prices across a continuous range. No model checker can enumerate those states. A clean pass means "no violation found in the explored region," not "no violation exists."

2.2 Level 2: Prove It for All Sizes (Theorem Proving)

The second approach is mechanized proof. We implemented the engine in Lean 4—a programming language that also serves as a proof assistant—and wrote a machine-checked proof that the engine's key guarantee holds not for small configurations, but for *all possible inputs of all sizes.*

A Lean proof is verified by the proof checker on every build. There is no "trust me, I checked it carefully." Either the proof compiles or it does not. When it compiles, the guarantee is unconditional within the stated preconditions.

This process discovered two further issues: the matching loop's original termination bound was an arbitrary constant, which made the key theorem false for large books, and the later full-invariant proof exposed an MTL/minimum-quantity routing error in the executable transcriptions. The fuel issue required a universally quantified statement—bounded model checking had never reached configurations large enough for the constant to matter.

2.3 What We Found

Finding	Tool	Description
1	Bounded model checking (TLC)	FIFO violation: triggered stop orders retain their original submission timestamp, producing a non-monotonic queue when the triggered order rests
2	TLA+ modeling / specification review	Iceberg stranding: STP DECREMENT can drive visible quantity to zero without triggering iceberg reload, permanently stranding the order; the checked TLA+ model includes the fix, so this is reported as a prose-specification gap rather than a TLC counterexample
3	Theorem proving (Lean 4)	Unsound fuel bound: the matching loop's constant limit of 100 steps makes the preservation theorem false for books with more than 100 contra-side orders
4	Full-invariant proof (Lean 4)	Transcription defect: a passing MinQty pre-check could route a MARKET-TO-LIMIT order directly to dispose before conversion, allowing it to rest as MTL; the prose specification was correct, but both executable transcriptions diverged
5	TLA+-trace conformance testing	C++ dispose defect: a triggered STOP_MARKET order retaining DAY/GTD time-in-force could rest a market remainder on the book, violating NoRestingMarkets

2.4 What Each Layer Establishes

The three methods verify different objects and deliver different kinds of guarantee. Because "verified" and "proved" are easy to over-read, Table 1 states precisely what is established, by which artifact, and—equally important—what is not.

Table 1: The assurance boundary. Each layer verifies a distinct object; the guarantees do not compose into an end-to-end machine-checked refinement from prose specification to C++.

Layer	Object verified	Guarantee
TLC (TLA+)	The TLA+ model	Exhaustive, but only within the finite configurations of Table 6 (up to 2–3 orders, 2–3 prices). No claim beyond those bounds.
Lean 4	The Lean executable reference implementation	The full §13 book-state invariant suite is preserved for *arbitrary* finite books, under preconditions discharged by well-formedness. Machine-checked on every build.
Conformance testing	The C++ implementation	Agreement with the TLA+ model on projected observables, for the traces actually replayed (§5.1). Testing, not proof.
Not established	Lean reference ↔ C++ engine	No general refinement proof links the proved Lean reference to the shipped C++ engine; the two are connected only by conformance testing.

The Lean guarantee is unbounded but concerns the Lean reference; the C++ guarantee concerns the shipped engine but is bounded by the traces replayed. Neither substitutes for the other, and no layer claims a machine-checked path from the prose specification all the way to C++.

3 What Model Checking Found

3.1 Bug 1: The Stale Timestamp

The scenario. A stop order is submitted early—say at time 2—with a stop price of $1. It goes dormant, waiting for the last trade price to reach $1. Meanwhile, another order arrives at time 3 and rests on the bid side at $1.

Later, a trade occurs at $1, triggering the stop order. The stop converts to a live limit order and enters the matching pipeline. It does not fill immediately (the ask side is now empty), so it rests on the bid side at $1. It is appended to the back of that price level's queue—behind the order that arrived at time 3.

The queue at price $1 now reads: [order at time 3, order at time 2]. But time 3 > time 2, so the queue is not in ascending timestamp order. FIFO is violated.

The exact trace. TLC produced the following minimal counterexample:

```
Step 1: Submit SELL LIMIT  qty=1, price=1      (id=1, ts=1)
        -> Rests on ask:  askQ[1] = [order1(ts=1)]

Step 2: Submit BUY STOP_LIMIT  stop=1, price=1, qty=1  (id=2, ts=2)
        -> lastTradePrice = NULL -> does not trigger
        -> Placed in stops = {order2(ts=2)}

Step 3: Submit BUY LIMIT  qty=2, price=1       (id=3, ts=3)
        -> Matches askQ[1]: fills order1 fully (fillQty=1)
        -> Trade: price=1, aggressor=3, passive=1
        -> lastTradePrice = 1
        -> order3: remainingQty=1, rests on bid
           bidQ[1] = [order3(ts=3)]
        -> Stop cascade: lastTradePrice(1) >= stopPrice(1)
           -> order2 triggers, converts to BUY LIMIT
           -> Timestamp NOT updated: ts=2 (STALE)
           -> Ask side empty, no fill
           -> Dispose: order2 rests on bid
              bidQ[1] = [order3(ts=3), order2(ts=2)]

        FIFOWithinLevel check:
            bidQ[1][1].timestamp = 3
            bidQ[1][2].timestamp = 2
            3 < 2 ?  FALSE  ->  INVARIANT VIOLATED
```

Figure 1: TLC counterexample trace for the FIFO violation. The triggered stop order (id=2) retains its original timestamp (ts=2) and is appended behind an order with a later timestamp (ts=3), producing a non-monotonic queue.

Why manual review missed this. The prose specification treats stop triggering (§10.1) and order insertion (§6.1) as separate sections. Each section is locally correct. The interaction—that converted stops carry stale timestamps into insertion—is invisible when reading either section in isolation. TLC holds all three simultaneously by construction.

The fix. One line in the stop-trigger procedure: assign a fresh timestamp before re-entering the pipeline.

```
converted == [ConvertStop(s) EXCEPT !.timestamp = tm]
```

In prose: after converting a stop to its base type and before calling `PROCESS`, assign the stop a fresh timestamp reflecting the current time. This ensures the triggered order's queue priority reflects when it entered the live matching pipeline, not when it was originally submitted.

3.2 Bug 2: The Stranded Iceberg

The scenario. A resting iceberg order has total quantity 2 and display quantity 1, so its visible slice is 1 and its hidden reserve is 1. An incoming order from the same STP group arrives with STP policy DECREMENT.

DECREMENT reduces both sides by the minimum of the two visible quantities, without generating a trade. It reduces the resting order's visible quantity by 1 (to 0) and its remaining quantity by 1 (to 1). The incoming order is consumed and cancelled.

The resting order now has visible quantity 0 and remaining quantity 1. It should reload its visible slice from its hidden reserve. But the reload logic is only specified in the fill path. DECREMENT is not the fill path. The reload never triggers.

The result is a permanently stranded order: visible quantity 0 means `fillQty = min(incoming, 0) = 0`, so matching never touches it. It sits on the book indefinitely, occupying a queue slot, blocking nothing, and unreachable except by explicit cancel.

The illustrative state.

```
Resting iceberg: qty=2, displayQty=1, visibleQty=1,
                 remainingQty=2, stpGroup="G1"        (id=R)
Incoming:        qty=1, stpGroup="G1", stpPolicy=DECREMENT  (id=I)

DECREMENT:
    reduceQty = min(1, 1) = 1
    I.remainingQty = 0   -> I is cancelled
    R.visibleQty   = 0
    R.remainingQty = 1   -> R remains on book

Post-decrement state of R:
    visibleQty=0, remainingQty=1, displayQty=1
    -> Reload path not invoked
    -> R sits on book, invisible, unreachable by matching
```

Note on claim level. This scenario is reachable in the prose specification as written. However, the TLA+ model as checked incorporates the fix because the stranded state would cause the model checker to loop: subsequent orders would find visible quantity 0 and skip the order without decrementing fuel, creating an effective infinite loop in the matching operator. The bug is presented as a specification gap—a path the spec silently omits—rather than as a TLC invariant violation with a counterexample trace.

The fix. After DECREMENT, check whether the resting order is an iceberg with visible quantity 0 and hidden reserve remaining. If so, reload:

```
ELSE IF newRest.visibleQty = 0 ∧ newRest.displayQty /= NULL THEN
    LET reloadQty == Min(newRest.displayQty, newRest.remainingQty)
        reloaded  == [newRest EXCEPT !.visibleQty = reloadQty,
                                     !.timestamp = tm,
                                     !.status = "PARTIAL"]
        sc == SetContra(side, bQ, aQ, bestP,
                        restTail \o ⟨reloaded⟩)
    IN DoMatch(newInc, sc.bQ, sc.aQ, tds, tm + 1, fuel - 1)
```

This mirrors the reload behavior in the normal fill path, extending it consistently to the DECREMENT path.

3.3 What Else We Noticed

Three additional interactions were noted during model exploration as potential specification ambiguities. No invariant violation was produced for them; they are reported as design observations.

FOK combined with STP DECREMENT creates an ambiguity: the FOK pre-check counts non-conflicting visible quantity to ensure full fill is possible, but DECREMENT can consume the incoming order's quantity without generating trades. Whether the result constitutes a FOK violation depends on whether "fully filled" means "fully traded" or "remaining quantity reaches zero"—the spec does not address this case. Similarly, MinQty combined with STP DECREMENT may produce a situation where the minimum-quantity pre-check passes but the actual traded quantity falls below the minimum because DECREMENT consumed quantity before any fills occurred. Finally, the spec clears `minQty` in two separate places (the MTL path

and the normal matching path)—a duplication that is currently correct but fragile under any future addition of routing paths.

3.4 The Scope and Limits of Model Checking

The configurations we explored generated 36 million distinct states. This is large enough to find the feature-interaction bugs above. It is not large enough to reason about production-scale books.

A production matching engine may hold thousands of resting orders per price level. Model checking with 2 orders, quantities up to 2, and 3 prices cannot say anything about whether a matching loop terminates correctly with 200 orders at one price. For that kind of guarantee, we need a proof.

4 The Lean Proof—Key Ideas

TLC searches a bounded state space. Production engines hold millions of orders, far beyond any configuration TLC can enumerate. To establish correctness at arbitrary scale, we translated the fixed specification into Lean 4 and proved the key preservation property mechanically.

4.1 The Goal

We want to prove: after processing any valid order on any valid book, the book is still uncrossed and sorted on both sides. For any number of orders, any book size, forever.

The formal statement defines the combined invariant

$$\begin{aligned}\texttt{AllInv}(b) \;\equiv\; & \texttt{BookUncrossed}(b) \;\wedge \\ & \texttt{bidsSortedDescB(b.bids) = true} \;\wedge \\ & \texttt{asksSortedAscB(b.asks) = true},\end{aligned}$$

and the main theorem is:

```
theorem processOrder_preserves_AllInv
    (fuel : Nat) (o : Order) (b : BookState)
    (hpok : OrderProcOk o)
    (hstops : StopsNoPostOnly b)
    (h : AllInv b) :
    AllInv (processOrder fuel o b).book
```

The sortedness conjuncts are not cosmetic. `BookUncrossed` alone is not sufficient for the induction to close: the induction hypothesis needs to know that both sides of the book are monotone in price. This was established empirically—the proof refused to close under the weaker hypothesis, at which point the invariant was strengthened and the proof proceeded. The proof refined the invariant.

4.2 The First Surprise: The Theorem Was False

Lean's termination checker requires every recursive function to be provably total. The matching loop `doMatch` is recursive, so the original implementation passed the same explicit *fuel* argument into the matching loop, using a constant bound:

```
def defaultFuel : Nat := 100
def doMatch (fuel : Nat) ... : MatchResult :=
  match fuel with
  | 0       => /* fuel exhausted: return as-is */
  | fuel'+1 => /* one step, recurse with fuel' */
```

Using this constant as the matching-loop bound hides an assumption: matching always completes within 100 steps. That assumption does not hold at arbitrary scale. Consider a book with 200 resting asks at price $50 and one incoming buy at price $100. `doMatch` consumes the first 100 asks, exhausts its fuel, and returns—with the incoming order still having remaining quantity and 100 asks still resting at price $50. The disposition phase then inserts the buy at $100 on the bid side. Now the best bid is $100 and the best ask is $50. The book is crossed. `AllInv` is false.

This is precisely where bounded model checking and theorem proving diverge. TLC's explored configurations never exceeded a handful of orders per side, so the constant bound was never a binding constraint. Only the universally quantified Lean statement forced the question: is the fuel bound sufficient, or is it just an assumption?

4.3 The Core Insight: Every Step Makes Progress

The fix is to compute the fuel bound from the book state and prove that it is always sufficient:

```
def matchMeasure (contra : List PriceLevel) (_inc : Order) : Nat :=
    totalRemaining contra + orderCount contra + contra.length

def computeMatchFuel (b : BookState) (side : Side) : Nat :=
    matchMeasure (contraLevels b side) dummyInc + 1
```

The measure has three components. `totalRemaining`—the sum of all remaining quantities on the contra side—accounts for fills. `orderCount`—the number of individual orders—accounts for orders that are skipped without a fill (STP cancellations, zero-visible iceberg slots). `contra.length`—the number of price levels—accounts for level exhaustion steps that remove an empty level.

The key technical lemma is the *progress lemma*: every non-terminal step of `doMatch` strictly decreases this measure. Table 2 gives the branch-by-branch analysis.

Table 2: Progress analysis for `doMatch` branches. Terminal branches stop the loop; measure-decreasing branches recurse on a strictly smaller state.

Branch	Kind	Why the measure decreases
Full fill (resting consumed)	Decrease	`orderCount` drops by 1
Partial fill (incoming done)	Terminal	$remainingQty = 0$
STP cancel newest	Terminal	incoming cancelled
STP cancel oldest	Decrease	one order removed
STP cancel both	Terminal	incoming cancelled
STP decrement	Decrease	resting qty strictly reduced
Iceberg fill + reload	Decrease	fill reduced `totalRemaining` before reload
Level exhausted	Decrease	empty level removed
Zero-visible skip	Decrease	one order removed
No crossable orders	Terminal	$\neg$ `canMatchPrice`

The iceberg reload case is the subtlest. Reload appears to "add liquidity back": the resting order's visible quantity jumps from zero to the display quantity. But `totalRemaining` is not affected by this redistribution—only the visible/hidden split changes—and the fill step that triggered the reload strictly decreased `totalRemaining` by the filled quantity. The composition is a net decrease.

4.4 The Elegant Argument

Once the constructive proof of the full pipeline was complete, a second shorter proof was developed for the matching-and-dispose core, in the file `TheoremsElegant.lean` (311 lines, 6 theorems).

The matching loop continues as long as the incoming order's price is willing to cross the best price on the opposite side—a condition captured by the predicate `canMatchPrice` (defined formally in §7.3). When the loop stops, this condition is false.

The insight is that when `doMatch` terminates on a fuel-sufficient run, the resulting state falls into exactly one of three cases:

1. **Aggressor consumed.** Remaining quantity is zero (or the order was cancelled). The incoming order is gone. `doMatch` only removes orders from the contra side, never adds to it; removing orders cannot create a cross.
2. **Contra side empty.** Nothing left on the opposite side to cross against. An empty side is vacuously non-crossing.
3. **Non-crossing head.** Both sides are non-empty but the contra head does not cross the incoming price. This is the only way `doMatch` can stop with both sides active: if prices crossed, the loop guard `canMatchPrice` would fire and matching would take another step. Since it stopped, prices do not cross.

There is no fourth case. A state where both sides are active and still crossing is not a terminal state—fuel sufficiency rules it out. The three cases compose with the disposition step directly: Case 1 returns the book unchanged, Case 2 inserts on an empty contra side (vacuously non-crossing), Case 3 inserts at a price strictly separated from the contra head.

This 311-line proof treats `doMatch` as a black box characterized by its termination condition. It reuses the constructive proof's fuel-sufficiency lemma as an imported fact. The contrast between the two proof styles is instructive:

	Constructive	Structural
File	`Theorems.lean`	`TheoremsElegant.lean`
Lines	3,965	311
Theorems/lemmas	74	6
Argument	Branch-by-branch	Three-case analysis

The constructive proof discharges fuel sufficiency, sortedness preservation, side isolation, and the mutual pipeline theorem. The structural proof reuses those lemmas and shows that, once the termination condition is established, the matching-and-dispose argument itself is short.

4.5 What Else the Proof Discovered

Two things were discovered during the proof that were not in the original specification's list of things to worry about.

Sortedness is load-bearing. The prose specification states `BookUncrossed` as the key invariant. The Lean proof required `AllInv`—uncrossed plus sorted on both sides—because the induction step on the matching loop needed to know that the contra side was monotone in price to establish that the residual order's price was separated from the contra head. Sortedness is not a cosmetic property.

Well-formedness constraints are discovered, not guessed. The proof required two fragments of the specification's well-formedness rules as explicit preconditions: post-only orders have a defined price, and stop orders are not post-only. Neither was included in the initial proof attempt. The proof failed at specific branches, and the failures pointed precisely to these missing hypotheses. The proof told us which constraints were load-bearing. The precise Lean predicate and its derivation from the specification's well-formedness constraints are given in §7.2.

5 From Specification to Implementation

The fixed specification—prose amended by the two bug fixes described in §3—is designed to serve as the primary input for a systems-language implementation. The specification's phase structure, well-formedness constraints, and invariant definitions provide a precise blueprint: each processing phase maps directly to an implementation function, each well-formedness constraint maps to an admission check, and each invariant maps to a runtime assertion.

For the Lean reference implementation, the gap between specification and implementation is closed by the proof itself: the code that tests run against is the same code the theorems reference. There is no extraction layer, no refinement obligation.

For a systems-language implementation such as C++, the gap can be narrowed by conformance testing against the TLA+ model. In this approach, TLC generates execution traces—sequences of orders and the model's expected book state after each—and a test harness replays these traces against the implementation, comparing its output state-for-state against the model. This pipeline does not constitute a refinement proof, but it provides a level of assurance substantially above conventional unit testing by covering the same combinatorial feature interactions that TLC explores.

5.1 The Conformance Pipeline

We implemented this approach for a header-only C++ matching engine (`matcher_stl/src/engine.h`, ∼645 source lines). The pipeline has three stages. First, TLC is invoked in simulation mode against seeded configurations of each of twelve scenarios—empty, shallow and deep book, iceberg-dominant, mixed time-in-force, stops-pending, self-trade-prevention, and so on—with a depth bound that keeps each random walk short enough to terminate. Second, a converter parses each trace and records, for every action, the spec's projected observables: top-of-book on each side, total book-order count, pending-stop count, last trade price, and the list of trades produced by that transition. Third, the C++ conformance harness replays each action against the engine and compares the engine's observables field-by-field to the projection after every step.

A sweep of approximately 232,000 traces across the twelve scenarios yielded roughly 1.5×10^6 per-step comparisons. Divergences point directly at the offending transition with a minimal repro that is typically fewer than ten actions long.

5.2 A Bug Conformance Testing Found

The scenario. A stop-market order is submitted with time-in-force DAY; its stop price is not yet met, so it is placed dormant in the stop set. Later, an amend re-prices a resting limit order across the spread; the amend executes a trade whose price satisfies the dormant stop's trigger condition.

The stop fires. Its type is rewritten from STOP_MARKET to MARKET; its time-in-force is unchanged, as specified. It enters the matching pipeline as a market order, fills partially against the remaining liquidity, and reaches the post-match *dispose* step with a non-zero remainder.

What the specification requires. The specification is unambiguous at this point: DISPOSE drops any market remainder regardless of time-in-force, and safety invariant INV-8 NORESTINGMARKETS forbids a market order from appearing in either book queue. No market order may survive past the dispose step.

The defect. The C++ `Dispose` function gated the market-drop clause on the GTC branch only; the DAY and GTD branches fell through to the book-insertion path, pushing the leftover market quantity onto the book at price zero. From that point every subsequent action carried

an extra phantom order visible through the book's aggregate count, and later aggressors could trade against the zero-priced order, spuriously producing additional fills.

The correction lifts the market-drop guard to apply to all three persistent time-in-force branches. The fix preserves the existing behaviour for user-submitted market orders—which well-formedness constrains to IOC/FOK—and adds the spec-required coverage for triggered stops, whose converted form retains the original time-in-force.

Why conventional testing missed it. The engine had passed sixty-five directed unit tests covering limit matching, iceberg reload, FOK feasibility, stop triggering, and partial fills in isolation, but none exercised a stop-triggers-on-amend path whose triggered market had left-over quantity. A second oracle, a sibling C++ reference implementation shadowing the engine through random differential testing at the scale of tens of millions of operations per seed, also did not surface the defect: its own `dispose` function contained the identical missing market guard on the DAY/GTD branches, and a differential test cannot see a bug both implementations share. The formal specification, by contrast, treated DAY/GTD uniformly with GTC in the dispose clause and made a resting market a safety violation. Comparing the engine's behaviour against the spec's projected state was the first oracle positioned to distinguish the two.

This is the structural argument for spec-based conformance testing over inter-implementation differential testing. An independent C++ reference catches all data-structure bugs that do not appear in both codebases, but it cannot catch a semantic error that both codebases inherited from a shared mental model—only a specification rooted outside of C++ can.

6 Discussion

6.1 Feature Composition Is Where Bugs Hide

Both model-checking findings share a structural pattern: each involved feature is correctly specified in isolation, but the specification omits a clause covering their interaction. Stop orders are correctly described; iceberg reload is correctly described; FIFO insertion is correctly described. None of these sections contains an error when read alone. The bug lives in the intersection: what happens to an iceberg order's timestamp when the mechanism that zeros its visible quantity is not the fill path but the STP DECREMENT path?

This is not a failure of careful writing. It is a structural limitation of prose specifications. Prose sections have natural scope boundaries. Interactions between features that span sections are systematically harder to see than behaviors within a single section.

A formal model collapses all features into a single operational semantics and evaluates invariants globally. The intersection cases are not special—they are just more states.

A more robust specification of iceberg reload would state the reload condition declaratively as an invariant on resting order state: "a resting order must not have visible quantity = 0 and remaining quantity > 0." Rather than specifying reload as a step triggered by the fill path, it would be specified as a consequence of this invariant: any action that could produce this state must be followed by a reload. This declarative framing would make the omission in DECREMENT immediately apparent.

6.2 Model Checking and Theorem Proving Are Complementary

TLC and Lean surfaced different classes of issue. TLC produced the two feature-composition defects as concrete multi-step traces within minutes of starting the exploration. A proof attempt would not have generated those traces directly; the proof effort is focused on establishing what holds, not on constructing witnesses for what fails.

Lean's universal quantification, in turn, made the constant-fuel assumption visible. TLC did not reach configurations where the fuel bound mattered. Only the universally quantified statement forced the question.

The cost profiles differ dramatically. TLC returned its first counterexample in 102 seconds of wall time. The Lean proof accumulated several weeks of interactive development and produced a 5,699-line artifact. The two are complementary rather than substitutable: TLC's speed is useful early in specification work; Lean's guarantees scale beyond any bounded configuration.

6.3 The Role of AI

The TLA+ model and the Lean 4 proofs were developed interactively with a large language model acting as a coding assistant. Strategic decisions—the progress-lemma decomposition, the three-case analysis in the elegant proof, and the side-parameterization refactor—were human contributions. The bulk of tactic scripting and operator translation was produced by the model under that guidance.

We mention this because the per-file size and per-lemma count reported in the paper reflect this mode of work. The methodology described is independent of whether the implementer is a single engineer, a team, or an AI-augmented workflow. Neither layer alone would have produced the artifact: the strategic decompositions guided the model to close proofs it would not have closed autonomously, and the model produced the thousands of lines of case-splitting and arithmetic invocations that the human-directed strategy required.

6.4 Cost and Value

TLA+ and TLC took approximately two days of work and produced one concrete invariant counterexample plus one specification gap. The Lean proof took approximately four weeks and produced a universal guarantee plus discovered the fuel-bound defect and, during the later full-invariant extension, an MTL/minimum-quantity transcription defect. These have different return-on-investment profiles.

For a specification that is still under active development, TLC's rapid feedback is most valuable: a two-day investment finds concrete counterexamples before any code is written. For a specification that is considered stable and whose correctness matters at production scale, Lean's universal guarantee is the appropriate next step. The two levels are designed to be used in sequence, not as alternatives.

7 Technical Details

7.1 The Specification Source

The object of study is an internal prose specification, `matching-engine-formal-spec.md` version 1.2.0, describing a generic asset-agnostic order matching engine with price-time priority. The specification is 979 lines of structured pseudocode covering primitive domains, order well-formedness constraints, the order book structure, a core matching algorithm, post-match disposition, five order types, four STP policies, iceberg reload semantics, stop triggering with cascading evaluation, cancel, and amend. It was written with care: well-formedness constraints are enumerated, invariants are stated formally, and edge cases such as MTL price derivation and FOK dry-run semantics are treated explicitly.

7.2 Well-Formedness Constraints

The prose specification enumerates 23 well-formedness constraints (WF-1 through WF-20 plus three a/b suffixed refinements) defining the valid inputs to the engine. Table 3 summarizes them.

Table 3: Well-formedness constraints (excerpt from the prose specification). TLA+ `WellFormed` omits WF-6/7, WF-11/12, and WF-17; `MakeOrder` initializes the omitted `remainingQty`/`visibleQty` fields. Lean `Order.wellFormed` omits WF-6/7, WF-12, and WF-17; it includes WF-11 but not WF-12. Lean's `OrderProcOk` requires only the two load-bearing fragments (WF-13 and its corollary).

ID	Group	Constraint
WF-1	Qty	$quantity > 0$
WF-2/2a/2b	Price	LIMIT, STOP_LIMIT require $price > 0$; MTL has $price = \bot$
WF-3	Price	MARKET, STOP_MARKET have $price = \bot$
WF-4	Stop	Stop orders require $stopPrice > 0$
WF-5	Stop	Non-stop orders have $stopPrice = \bot$
WF-6/7	TIF	GTD requires $expireTime > now$; otherwise $expireTime = \bot$
WF-8/8a	TIF	MARKET⇒IOC/FOK; MTL⇒GTC/DAY
WF-9/10	Iceberg	$0 < displayQty \leq quantity$; icebergs must be LIMIT
WF-11/12	Iceberg	Initial $remainingQty = quantity$; $visibleQty = \min(displayQty, quantity)$
WF-13	PostOnly	Post-only⇒LIMIT
WF-14	PostOnly	Post-only$\not\Rightarrow$IOC/FOK
WF-15	PostOnly	MARKET, MTL⇒¬post-only
WF-16	STP	$selfTradeGroup = \bot \iff stpPolicy = \bot$
WF-17	STP	$stpPolicy \in \{$CANCEL_NEWEST, CANCEL_OLDEST, CANCEL_BOTH, DECREMENT$\}$
WF-18	MinQty	$0 < minQty \leq quantity$
WF-19	MinQty	MinQty⇒¬post-only
WF-20	MinQty	FOK⇒$minQty = \bot$

The Lean predicate `OrderProcOk` used in the main theorem is the minimum subset of these constraints that the matching loop actually depends on:

```
def OrderProcOk (o : Order) : Prop :=
    (o.postOnly = true -> o.price.isSome = true) /\
    ((o.orderType = .stopLimit \/ o.orderType = .stopMarket) ->
       o.postOnly = false)
```

The first conjunct rules out the post-only / no-price configuration (WF-13: post-only ⇒ LIMIT; WF-2: LIMIT ⇒ price defined). The second conjunct—stop orders are not post-only—is a consequence by contrapositive. Both were discovered during the proof: earlier attempts omitted them and branches of the matching loop that case-split on an undefined price failed. The proof told us which WF clauses were load-bearing.

7.3 Invariant Suite and Loop Guard

The prose specification states 14 invariants that must hold after every `PROCESS` call. Table 4 gives the book-structural core; Table 5 gives the complete TLC-checked inventory.

Table 4: Core book invariants.

ID	Group	Statement
INV-1	Structural	Every price level has a non-empty order queue
INV-2	Structural	$\forall$ consecutive bid levels (L_1, L_2): $L_1.price > L_2.price$
INV-3	Structural	$\forall$ consecutive ask levels (L_1, L_2): $L_1.price < L_2.price$
INV-4	Structural	$bestBid < bestAsk$ or either side empty (**uncrossed**)
INV-5	Content	Every resting order has $remainingQty > 0$
INV-6	Content	Every resting order has status $\in \{\text{NEW}, \text{PARTIALLY\_FILLED}\}$
INV-7	Ordering	FIFO within each price level: consecutive orders have strictly increasing timestamps
INV-8	Content	No MARKET orders rest on the book
INV-9	Trade	Every trade executes at the passive order's price
INV-10	Events	Events emitted in causal order
INV-11	PostOnly	No trade where the aggressor was post-only
INV-12	STP	No trade between orders in the same STP group
INV-13	Content	No MTL orders rest on the book
INV-14	Content	No $minQty$ is set on any resting order

Table 5: Invariants checked by TLC and Lean.

ID	TLA+ Operator	Description	TLC	Lean
INV-1	`NoEmptyLevels`	Every price level has a non-empty order queue	trivial	✓
INV-2	(sortedness)	Bids strictly descending	by constr.	✓
INV-3	(sortedness)	Asks strictly ascending	by constr.	✓
INV-4	`BookUncrossed`	$bestBid < bestAsk$ or either side empty	✓	✓
INV-5	`NoGhosts`	Every resting order has $remainingQty > 0$	✓	✓
INV-6	`StatusConsistency`	Resting orders have status NEW or PARTIAL	✓	✓
INV-7	`FIFOWithinLevel`	FIFO within level (strictly increasing timestamps)	fixed*	✓
INV-8	`NoRestingMarkets`	MARKET orders never appear on the book	✓	✓
INV-9	(construction)	Trade price equals the passive order's price	by constr.	by constr.
INV-11	`PostOnlyGuarantee`	No trade where the aggressor was post-only	✓	✓
INV-12	`STPGuarantee`	No trade between orders in the same STP group	✓	✓
INV-13	`NoRestingMTL`	MTL orders are converted before resting	✓	✓
INV-14	`NoRestingMinQty`	$minQty$ is cleared before an order rests	✓	✓

IDs match the canonical §13 summary of the prose specification. TLC column: ✓ means bounded exhaustive pass across all configurations in Table 6. Lean column: ✓ means mechanically proved for all book sizes—INV-2/3/4 as part of `AllInv`, the book-state invariants via the capstone `process_preserves_BookInvariant`, and INV-11/12 via `process_PostOnlyGuarantee` and `process_STPGuarantee`. INV-9 holds by construction; INV-10 (event ordering) is not modeled in Lean. *INV-7 was violated under the original specification and passes after the fix described in §3.1.

A single predicate governs the matching loop's continuation:

$$\texttt{canMatchPrice}(o, p) \equiv \begin{cases} \top & o.type \in \{\text{MARKET}, \text{STOP\_MARKET}\} \\ o.price \geq p & o.side = \text{BUY},\ o \text{ priced} \\ o.price \leq p & o.side = \text{SELL},\ o \text{ priced} \\ \top & o.type = \text{MTL},\ \text{no trades yet} \\ \text{derived price} & o.type = \text{MTL},\ \text{after first fill} \end{cases}$$

This predicate is the loop guard of `doMatch`. When the loop terminates in a non-empty contra side, it terminates because $\neg\texttt{canMatchPrice}(o, bestContra)$. This is the seed of the three-case elegant proof: if matching stopped because prices no longer cross, then the book cannot be crossed after disposition places the residual on the same side as the aggressor.

7.4 TLA+ Model Structure

The TLA+ specification (`MatchingEngine.tla`, approximately 950 lines) is a direct translation of the prose specification. The state consists of eight variables:

```
bidQ          : PRICES → Seq(Order)    (* per-price bid queues *)
askQ          : PRICES → Seq(Order)    (* per-price ask queues *)
stops         : Set(Order)              (* dormant stop orders *)
```

```
lastTradePrice : PRICES \cup {NULL}     (* last execution price *)
postOnlyOK     : Bool                   (* no post-only aggressor
   trade *)
stpOK          : Bool                   (* no same-group trade *)
nextId         : Nat                    (* next order ID *)
clock          : Nat                    (* logical clock *)
```

The model has four actions: `SubmitOrder`, `CancelOrder`, `AmendOrder`, and `TimeAdvance`. `SubmitOrder` nondeterministically selects all valid order parameters and invokes the full processing pipeline. TLC's exhaustive exploration therefore considers every valid combination of order type, side, price, quantity, TIF, STP policy, iceberg flag, post-only flag, and minimum quantity, within the configured bounds.

The model is parameterized by `PRICES`, `MAX_QTY`, `MAX_ORDERS`, and `MAX_CLOCK`. The configurations explored are shown in Table 6.

Table 6: Model checking configurations and statistics.

Configuration	Orders	Qty	Prices	Amend	States Gen.	Distinct	Runtime	Result
Tiny	2	1	{1,2}	No	2,979,719	1,123,333	20 s	Pass
Small	2	2	{1,2}	No	11,528,343	6,037,674	1 m 42 s	Pass
Medium	2	2	{1,2,3}	No	36,700,016	21,261,901	5 m 34 s	Pass
With Amend	2	2	{1,2}	Yes	25,209,607	9,104,902	3 m 33 s	Pass
3-order	3	2	{1,2}	No	>45 M	>26 M	>10 m	*partial*

All "Pass" results above are post-fix. Prior to fixing the stop-timestamp bug, the Small configuration produced a counterexample on the `FIFOWithinLevel` invariant within the first 11.5 million states. The 3-order run was interrupted before state-space exhaustion; "partial" means no violation was found in the explored states, not clean passage under all 3-order behaviors.

The model does not cover: multiple STP group identifiers (only `"G1"` is used); GTD expiry without wall-clock constraints; event ordering (INV-10); or implementation correctness as opposed to specification consistency.

7.5 Lean Proof Architecture

The Lean proof artifacts are summarized in Table 7.

Table 7: Lean 4 proof artifacts (final state).

File	Lines	Theorems	Proof style
`Theorems.lean`	3,965	74	Constructive, branch-by-branch
`TheoremsFull.lean`	1,423	77	§13 book-state suite + trade guarantees
`TheoremsElegant.lean`	311	6	Three-case trichotomy
Total	**5,699**	**157**	Complementary proof artifacts

`Theorems.lean` and `TheoremsElegant.lean` establish the `AllInv` core—uncrossed plus sorted on both sides. `TheoremsFull.lean` lifts the remaining §13 book-state invariants (no empty levels, no ghosts, status consistency, FIFO, and no resting market/MTL/MinQty orders) together with the post-only and STP guarantees on emitted trades, culminating in the capstone `process_preserves_BookInvariant`. Lifting INV-13 also surfaced a transcription defect: a well-formed MTL order carrying a `minQty` could come to rest mistyped as MTL at price zero, because the minimum-quantity pre-check short-circuited the MTL routing phase. The prose specification was correct; the Lean and TLA+ transcriptions had both diverged from it, and both are now fixed.

The mutual induction covers three functions—`processOrder`, `processCascade`, and `processTriggeredStops`—which are mutually recursive because a trade can trigger a dormant stop, which converts to a market order and re-enters the pipeline, which can in turn produce more trades and trigger more stops:

```
theorem process_all_preserve_AllInv : forall fuel,
  (forall o b, OrderProcOk o -> StopsNoPostOnly b -> AllInv b ->
    AllInv (processOrder fuel o b).book /\
    StopsNoPostOnly (processOrder fuel o b).book) /\
  (forall trades b, StopsNoPostOnly b -> AllInv b ->
    AllInv (processCascade fuel trades b).book /\
    StopsNoPostOnly (processCascade fuel trades b).book) /\
  (forall orders b, (forall o in orders, o.postOnly = false) ->
                    StopsNoPostOnly b -> AllInv b ->
    AllInv (processTriggeredStops fuel orders b).book /\
    StopsNoPostOnly (processTriggeredStops fuel orders b).book)
```

Induction proceeds on `fuel`. The base case (`fuel = 0`) is trivial because all three functions return the book unchanged. The step case discharges every phase of `processOrder`: stop handling, post-only rejection, FOK pre-check, MinQty pre-check, market-to-limit routing, normal matching, dispose, and stop cascade.

The subtlest sub-case is MTL Phase 4 case 3: a MARKET-TO-LIMIT order that fills partially, converts to a LIMIT order, then runs a second matching pass with the derived limit price. The second `doMatch` operates on the post-first-match book with a freshly constructed incoming order, and the non-crossing precondition for the subsequent dispose must be extracted from a `doMatch` call that does not match the form of any existing helper lemma. The resolution was to parameterize the non-crossing extraction helpers by the starting timestamp so they apply uniformly to both the first matching pass (which uses `b.clock + 1`) and the MTL second pass (which uses `mr.clock`).

Every matching lemma has both a buy-side and a sell-side form. The first proof attempt duplicated every lemma across sides. The fix was to parameterize each lemma by a `Side` value and perform `cases side` only at the leaves where the concrete direction of the inequality matters. After side-parameterization, the sell mirror of the fuel-sufficiency proof compiled on the first try.

7.6 The Capstone Theorem and Trusted Computing Base

The book-state guarantees combine into a single capstone theorem. In full Lean 4 syntax:

```
theorem process_preserves_BookInvariant
    (b : BookState) (o : Order)
    (hall : AllInv b) (hpok : OrderProcOk o) (hsnp : StopsNoPostOnly b)
    (hb : BookOk b) (hstops : StopsWF b) (hok : OrderRestOk o) :
    BookInvariant (process b o).book
```

Here `BookInvariant` bundles uncrossedness, no empty levels, no ghost orders, status consistency, FIFO within each level, and the no-resting market/MTL/MinQty conditions (INV-1 and INV-4 through INV-8, INV-13, INV-14); the sortedness conjuncts (INV-2, INV-3) travel with `AllInv`. The two trade-event guarantees are proved separately and *unconditionally*—an order reaches a matching phase only after failing the post-only pre-check, so every emitted trade satisfies them by construction:

```
theorem process_PostOnlyGuarantee (b : BookState) (o : Order) :
    PostOnlyGuarantee (process b o).trades      -- INV-11
theorem process_STPGuarantee     (b : BookState) (o : Order) :
    STPGuarantee     (process b o).trades       -- INV-12
```

Preconditions. The hypotheses are logical preconditions of this preservation step, not domain restrictions smuggled in to make the theorem true; each is discharged from ordinary inputs. `OrderRestOk` and `OrderProcOk` follow from the specification's well-formedness predicate (`OrderRestOk_of_WellFormed`). `BookOk` and `StopsWF` hold of the empty book (`BookOk_empty`, `StopsWF_empty`) and are preserved by `process` (`process_preserves_BookOk`), and `AllInv` is likewise inductive; so the whole precondition bundle holds of every book reachable from an empty start, and the invariant holds along every execution rather than at isolated states.

Trusted computing base. The proof rests on an unusually small base. The project builds with Lean 4 (toolchain `v4.26.0`) and has *no external dependencies*—in particular it does not use Mathlib; every module imports only other modules of the project. No proof file contains `sorry`, `admit`, `native_decide`, or any custom `axiom`. Printing the axiom footprint of the capstone confirms it:

```
'process_preserves_BookInvariant' depends on axioms:
    [propext, Classical.choice, Quot.sound]
'process_PostOnlyGuarantee' depends on axioms: [propext, Quot.sound]
'process_STPGuarantee'      depends on axioms: [propext, Quot.sound]
```

These are the three standard axioms of Lean's classical core; the trade guarantees do not even require choice. Every accepted term is checked by the Lean kernel, and no external decision procedure or SMT solver participates in acceptance. The AI-assisted development described in §6.3 therefore affects only the *cost* of producing the proof, not the basis on which it is trusted: a proof the kernel rejects does not build.

7.7 Traceability

Table 8 makes the chain from specification clause to TLA+ artifact to Lean artifact explicit for the clauses exercised in the main theorem.

Table 8: Spec → TLA+ → Lean traceability for the clauses touched by the main theorem. The Lean column refers to `Theorems.lean`, `TheoremsFull.lean`, or `Invariants.lean`.

Spec clause	TLA+ artefact	Lean artefact	Role
WF-13	`WellFormed` guard	`OrderProcOk` field 1	Main-theorem precondition
WF-13 (derived)	`WellFormed` guard	`OrderProcOk` field 2	Main-theorem precondition
INV-2/3	not separately checked[†]	`bidsSortedDescB`, `asksSortedAscB`	Discovered: needed for induction
INV-4	`BookUncrossed`	`BookUncrossed`, part of `AllInv`	Main-theorem conclusion
INV-5	`NoGhosts`	`FullBookInv`	Proved preserved
INV-6	`StatusConsistency`	`FullBookInv`	Proved preserved
INV-7	`FIFOWithinLevel`	`FullBookInv`	Proved preserved
INV-8	`NoRestingMarkets`	`FullBookInv`	Proved preserved
INV-11	`PostOnlyGuarantee`	`PostOnlyGuarantee`	Proved (no precond.)
INV-12	`STPGuarantee`	`STPGuarantee`	Proved (no precond.)
INV-13	`NoRestingMTL`	`FullBookInv`	Proved preserved
INV-14	`NoRestingMinQty`	`FullBookInv`	Proved preserved
`canMatch`	`CanMatchPrice`	`canMatchPrice`	Matching loop guard
PROCESS §12	`ProcessOrder` (11 phases)	`processOrder` (11 phases)	Mutual induction target

[†]Structural sortedness is not stated as a separate TLC invariant because TLA+ sequences are ordered by construction from a sorted key. In Lean the sortedness is explicit because book levels are a `List PriceLevel` and the induction needs to know both sides are monotone. This mismatch was discovered *during* the proof: `BookUncrossed` alone is not inductive, which forced the addition of `AllInv = BookUncrossed` ∧ `bidsSortedDescB` ∧ `asksSortedAscB` (with the Boolean predicates applied to the bid and ask level lists and required to equal `true`).

8 Related Work

This work sits at the intersection of several literatures: verified auction and matching algorithms, industrial model checking, mechanized systems verification, model-based conformance testing, and machine-checked termination. We survey each and locate our contribution relative to the closest prior art.

Verified matching engines and auctions. The closest prior art is a line of Coq formalizations of exchange matching by Sarswat, Singh, Natarajan, and Garg. They verify *double-auction* matching: an early result proves fairness, uniformity, and individual rationality for single-round call auctions [15], extended to multi-unit trade requests with uniqueness theorems that justify extracted regulatory checkers [16, 18], and most recently to an $O(n \log n)$ *continuous* double-auction matcher with a matching lower bound [17]. In industry, Imandra has been used to verify the matching logic and FIX connectivity of real exchanges and dark pools [19, 20, 21]. Our work is complementary on three axes. First, *object*: we prove invariant preservation across a stateful order book over time, rather than the input/output correctness of one matching round, so time priority (FIFO), resting, and re-entry become first-class. Second, *features*: our engine carries the interacting order-type suite that makes production matchers hard—icebergs, four STP policies, post-only, FOK, minimum quantity, and stop triggering—and the defects we report live precisely in their intersections. Third, *method*: we cross-check a bounded TLA+ model, an unbounded Lean proof, and a C++ implementation against one another, rather than extracting a single verified reference. Published accounts combining all of these—a continuous, feature-rich, price-time-priority engine with a mechanized arbitrary-size proof *and* implementation conformance—remain, to our knowledge, rare. Adjacent threads—market-design analysis

of frequent batch auctions [5] and smart-contract verification for decentralized exchanges [6, 7]—are orthogonal to the operational correctness of a central-limit-order-book engine.

TLA+ and model checking in industry. TLA+ has been applied at scale at Amazon Web Services [1, 22], at Microsoft (Azure Cosmos DB) [24], and at MongoDB [23], with a recent survey cataloguing a decade of industrial practice [25]. We follow the Newcombe et al. template—use TLC to explore a specification's state space and find interaction bugs before coding—but differ in domain (a financial matching engine rather than a distributed storage protocol) and in the follow-through to a machine-checked proof in Lean 4. MongoDB's "eXtreme Modelling" [23] is the closest methodological analogue to our conformance stage: it generates tests from a TLA+ model and checks an implementation's traces against it. We adopt the same spec-as-oracle idea and combine it with random differential testing.

Model-based and conformance testing. Testing an implementation against a formal model has a long theory, from Tretmans's IOCO conformance relation on labelled transition systems [26] to the model-based-testing taxonomy and tooling of Utting, Pretschner, and Legeard [27, 28]. Our conformance harness is a concrete offline-generation, online-execution instance: TLC counterexample and simulation traces become the tests, and the model-checked TLA+ specification is the oracle. We trade IOCO's generality for a mechanized reference model and a domain-specific projection of observables.

Mechanized proof of systems software. The largest machine-checked software artifacts—seL4 [2], CompCert [3], CertiKOS [4], FSCQ [36], and CakeML [38]—prove functional correctness or refinement and consumed person-decades of effort. IronFleet [34] is especially relevant: it pairs TLA-style state-machine refinement with Hoare-logic verification, much as we pair a TLA+ action model with a Lean invariant proof—though we keep the levels as independent cross-checks over a separately written C++ engine rather than a single refinement stack. Verdi [35] and Iris [37] represent, respectively, verified distributed implementations and a general invariant-based concurrent program logic. The present work is narrower in ambition and cost (weeks, not years): we prove preservation of the full §13 book-state invariant suite for the Lean reference implementation, and its complexity lives in feature composition rather than in concurrency or low-level memory semantics.

Termination and well-founded recursion. The device in §4.3—replacing an arbitrary fuel bound with a state-derived measure and a progress lemma—is a standard pattern for defining general recursive functions and proving them total, studied extensively for HOL provers [29, 30, 31, 9] and surveyed across the partiality/recursion design space—domain predicates, fuel and step-indexing, sized types, well-founded recursion [32, 33, 10]. Automated methods search for lexicographic or size-based orders [30]; our measure is instead a single problem-specific function of book state whose decrease is also the load-bearing soundness argument for the matching loop—so proving termination and proving the fuel bound sufficient are the same step, pushed through a per-branch progress argument over 14 interacting branches.

Differential and property-based testing. Differential testing—comparing independent implementations on random inputs [39]—underlies our shadow test, which cross-checks the engine against a vector-based reference. The technique is well established for compilers, from Csmith [40] to equivalence modulo inputs [41]. Property-based testing [8] and its foundational verified variant QuickChick [42] connect random testing to stated properties. The lesson of our C++ finding (§5.2) sharpens this literature: a differential test between two implementations cannot catch a semantic error both inherit from a shared mental model—only a spec-rooted oracle can.

LLM-assisted formal verification. A growing body of work uses large language models to draft proofs and drive proof search: hammer integration [43], informal-to-formal sketching [44], growing lemma libraries [45], large-scale synthetic Lean data [46], retrieval-augmented premise selection [11], whole-proof generation [12], and in-context proof agents [47]. These target *autonomous* tactic search. Our experience is complementary: the model did not close the proof autonomously; it wrote naturalistic Lean under human strategic direction. The progress-lemma decomposition, the side-parameterization refactor, and the three-case trichotomy were human contributions; the ∼5,700 lines of case-splitting and `omega` invocations were the model's. Neither layer alone would have produced the artifact, and—per §7.6—the Lean kernel checks every accepted term regardless of who or what drafted it.

Spec-to-implementation synthesis. Program synthesis from formal specifications is a classical research area [13]. Recent work on LLM-driven synthesis [14] has reached practical levels for small programs. The approach taken in this work stops short of synthesis or refinement proof. Instead, it carries the specification forward to a C++ implementation through conformance testing against TLC traces, which is weaker than a proof but strong enough to expose semantic defects shared by ordinary unit tests and a sibling C++ reference.

9 Limitations and Reproducibility

9.1 Bounded Exploration, Not Proof (for TLA+)

TLC performs bounded exhaustive state enumeration. A clean run means no invariant violation was found within the explored configurations. It does not mean no violation exists for larger configurations, more orders, or more price levels.

Specific gaps:

- The 3-order partial run did not complete state-space exploration.
- No configuration with more than two distinct STP groups was checked.
- GTD time-based expiry was checked only via the simplified `TimeAdvance` action, not against wall-clock constraints.
- The event-ordering invariant (INV-10) is not formalized in the model.

Lean closes the unbounded gap for the full §13 book-state suite. The Lean proof establishes `process_preserves_BookInvariant` for arbitrary book sizes and arbitrary order counts, under preconditions discharged by well-formedness and the empty-book base case. This covers the `AllInv` core (INV-2, INV-3, INV-4) together with no empty levels (INV-1), no ghosts (INV-5), status consistency (INV-6), FIFO within level (INV-7), no resting markets (INV-8), no resting MTL (INV-13), and no resting MinQty (INV-14); the post-only (INV-11) and STP (INV-12) guarantees on emitted trades are proved separately and unconditionally. Only INV-9 (trade price equals the passive price, which holds by construction) and INV-10 (event ordering, which is not modeled in Lean) lie outside the Lean development.

9.2 Specification vs. Implementation

For the Lean reference implementation, the gap is closed. The Lean 4 code that tests run is the same code the theorems reference—there is no separate model, no extraction layer, and no refinement obligation. The semantic gap between tested behavior and proved behavior is zero.

For the C++ implementation, a gap remains. The Lean proof does not transfer automatically to C++, and the TLA+-trace conformance pipeline compares projected observables rather than proving refinement. The conformance-testing approach described in §5 narrows this gap and found a real implementation defect, but it does not close the gap. A refinement proof connecting the Lean reference to the C++ implementation is a natural next step.

9.3 Reproducibility

All artifacts—the prose specification, the TLA+ model and configurations, the Lean 4 project, the C++ implementation and conformance harness, and the paper source—are available in a public Git repository at https://github.com/formally-verified-exchange/verified-matching-engine. The top-level `README.md` lists prerequisites and gives step-by-step commands for building the Lean proofs and running TLC against each configuration. The complete artifacts are:

- `matching-engine-formal-spec.md`—the prose specification.
- `matcher_tla/MatchingEngine.tla`—the TLA+ specification (∼950 lines).
- `matcher_tla/MatchingEngine.cfg` and the per-configuration files `MatchingEngine_*.cfg` for Table 6.
- `matcher_tla/REPORT.md`—model checking output and bug descriptions.
- `matcher_lean/`—the self-contained Lean 4 project. The source files live under `matcher_lean/MatchingEngine/`; the three proof files are `Theorems.lean` (3,965 lines, 74 theorems), `TheoremsFull.lean` (1,423 lines, 77 theorems), and `TheoremsElegant.lean` (311 lines, 6 theorems). All are built and type-checked by running `lake build` from inside `matcher_lean/` and are imported from the library root `matcher_lean/MatchingEngine.lean`.
- `matcher_stl/`—the C++ implementation, directed tests, shadow differential harness, TLA+ trace converter, conformance replay harness, and `BUG_LOG.md` documenting implementation defects found by the testing pipeline.

To reproduce the FIFO counterexample:

1. Revert the stop timestamp fix in `ProcessTriggeredStops`: replace `[ConvertStop(s) EXCEPT !.timestamp = tm]` with `ConvertStop(s)`.
2. Run TLC with the Small configuration (`MAX_ORDERS=2, MAX_QTY=2, PRICES={1,2}`, amend disabled).
3. TLC will find a violation of `FIFOWithinLevel` within the first several million states.

To reproduce the fuel-unsoundness issue (§4.2):

1. Revert `computeMatchFuel` in `matcher_lean/MatchingEngine/Process.lean` to the constant `defaultFuel := 100`.
2. Re-run `lake build`. The proof of `doMatch_preserves_AllInv` will fail: the measure argument no longer dominates the recursive call bound, and any book with more than 100 contra-side orders produces a concrete cross after one application of `process`.

10 Conclusion

We formalized a feature-rich matching-engine specification in TLA+ and ran bounded exhaustive exploration across several configurations totaling over 36 million distinct states. TLC identified one specification defect—a FIFO violation caused by a triggered stop order retaining its original submission timestamp—and one specification gap in the interaction between STP DECREMENT and iceberg visibility. The FIFO counterexample is a three-order sequence whose minimality is part of the finding: reading the stop, insertion, and FIFO sections of the prose specification individually does not expose it, but an operational model that composes all three does.

We then proved in Lean 4 that the processing pipeline preserves the full §13 book-state invariant suite—the `AllInv` core (uncrossed plus sorted on both sides) together with no empty levels, no ghosts, status consistency, FIFO within each level, and no resting market/MTL/MinQty orders, plus the post-only and self-trade-prevention guarantees on emitted trades—under preconditions discharged by well-formedness, for arbitrary book sizes. `Theorems.lean` (3,965 lines, 74 theorems) proves the `AllInv` preservation theorem branch-

by-branch; `TheoremsFull.lean` (1,423 lines, 77 theorems) lifts the remaining invariants to the capstone `process_preserves_BookInvariant`; and `TheoremsElegant.lean` (311 lines, 6 theorems) adds a shorter structural argument for the matching-and-dispose core via a three-case analysis of the matching termination condition. All three files are type-checked by every build and contain no open obligations. One issue surfaced during the proof effort itself: the original arbitrary fuel bound made the preservation statement false for sufficiently populated books. Replacing it with a state-derived measure and a progress lemma restores soundness.

The broader takeaway is that bounded model checking and mechanized theorem proving play complementary roles on the same specification. TLC is effective at surfacing concrete feature-composition defects inside small state bounds at speeds useful during early specification work. Lean provides guarantees that hold for arbitrary scale and, as a side effect, exposes pipeline issues that bounded exploration cannot reach. Neither tool substitutes for the other. The combination—a few days of model checking followed by weeks of theorem proving—represents a tractable investment for systems where feature interactions can move real money in the wrong direction.

All artifacts described in this paper—the prose specification, the TLA+ model and configurations, the Lean 4 project including all proof files, and the conformance-tested C++ implementation with its TLA+-driven trace-replay harness—are publicly available at the repository linked on the first page.

Acknowledgments

The TLA+ model and the Lean 4 proofs were developed interactively with the assistance of a large language model, using the formal specification as the primary input. We thank the anonymous reviewers for their comments.

References

[1] C. Newcombe, T. Rath, F. Zhang, B. Munteanu, M. Brooker, and M. Deardeuff. How Amazon Web Services Uses Formal Methods. *Commun. ACM*, 58(4):66–73, 2015.

[2] G. Klein et al. seL4: Formal Verification of an OS Kernel. In *Proc. ACM SOSP*, 2009.

[3] X. Leroy. Formal Verification of a Realistic Compiler. *Commun. ACM*, 52(7):107–115, 2009.

[4] R. Gu et al. CertiKOS: An Extensible Architecture for Building Certified Concurrent OS Kernels. In *Proc. USENIX OSDI*, 2016.

[5] E. Budish, P. Cramton, and J. Shim. The High-Frequency Trading Arms Race: Frequent Batch Auctions as a Market Design Response. *Quarterly Journal of Economics*, 130(4):1547–1621, 2015.

[6] K. Bhargavan et al. Formal Verification of Smart Contracts: Short Paper. In *Proc. PLAS*, 2016.

[7] I. Grishchenko, M. Maffei, and C. Schneidewind. A Semantic Framework for the Security Analysis of Ethereum Smart Contracts. In *Proc. POST*, 2018.

[8] K. Claessen and J. Hughes. QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. In *Proc. ACM ICFP*, 2000.

[9] P. Manolios and D. Vroon. Termination Analysis with Calling Context Graphs. In *Proc. CAV*, 2006.

[10] K. R. M. Leino. Developing Verified Programs with Dafny. In *Proc. ICSE*, 2013.

[11] K. Yang et al. LeanDojo: Theorem Proving with Retrieval-Augmented Language Models. In *Proc. NeurIPS*, 2023.

[12] E. First, M. Rabe, T. Ringer, and Y. Brun. Baldur: Whole-Proof Generation and Repair with Large Language Models. In *Proc. ESEC/FSE*, 2023.

[13] Z. Manna and R. Waldinger. A Deductive Approach to Program Synthesis. *ACM TOPLAS*, 2(1):90–121, 1980.

[14] N. Jain et al. Jigsaw: Large Language Models Meet Program Synthesis. In *Proc. ICSE*, 2022.

[15] S. Sarswat and A. K. Singh. Formally Verified Trades in Financial Markets. In *Proc. ICFEM*, LNCS 12531, pages 217–232, 2020.

[16] R. Natarajan, S. Sarswat, and A. K. Singh. Verified Double Sided Auctions for Financial Markets. In *Proc. ITP*, LIPIcs 193, pages 28:1–28:18, 2021.

[17] M. Garg and S. Sarswat. Efficient and Verified Continuous Double Auctions. In *LPAR-25 (Complementary Volume)*, 2024. Also arXiv:2412.08624.

[18] M. Garg, N. Raja, S. Sarswat, and A. K. Singh. Double Auctions: Formalization and Automated Checkers. *Journal of Automated Reasoning*, 69:17, 2025.

[19] G. O. Passmore and D. Ignatovich. Formal Verification of Financial Algorithms. In *Proc. CADE*, LNCS 10395, pages 26–41, 2017.

[20] G. Passmore, S. Cruanes, D. Ignatovich, et al. The Imandra Automated Reasoning System (System Description). In *Proc. IJCAR*, LNCS 12167, pages 464–471, 2020.

[21] G. O. Passmore. Some Lessons Learned in the Industrialization of Formal Methods for Financial Algorithms. In *Proc. FM*, LNCS 13047, pages 717–721, 2021.

[22] C. Newcombe. Why Amazon Chose TLA+. In *Proc. ABZ*, LNCS 8477, pages 25–39, 2014.

[23] A. J. J. Davis, M. Hirschhorn, and J. Schvimer. eXtreme Modelling in Practice. *Proc. VLDB Endowment*, 13(9):1346–1358, 2020.

[24] F. Hackett, J. Rowe, and M. A. Kuppe. Understanding Inconsistency in Azure Cosmos DB with TLA+. In *Proc. ICSE-SEIP*, pages 1–12, 2023.

[25] R. Bögli, L. Lerena, C. Tsigkanos, and T. Kehrer. A Systematic Literature Review on a Decade of Industrial TLA+ Practice. In *Proc. iFM*, LNCS 15234, pages 24–34, 2024.

[26] J. Tretmans. Model Based Testing with Labelled Transition Systems. In *Formal Methods and Testing*, LNCS 4949, pages 1–38, 2008.

[27] M. Utting, A. Pretschner, and B. Legeard. A Taxonomy of Model-Based Testing Approaches. *Software Testing, Verification and Reliability*, 22(5):297–312, 2012.

[28] M. Utting and B. Legeard. *Practical Model-Based Testing: A Tools Approach*. Morgan Kaufmann, 2007.

[29] A. Krauss. Automating Recursive Definitions and Termination Proofs in Higher-Order Logic. PhD thesis, Technische Universität München, 2009.

[30] L. Bulwahn, A. Krauss, and T. Nipkow. Finding Lexicographic Orders for Termination Proofs in Isabelle/HOL. In *Proc. TPHOLs*, LNCS 4732, pages 38–53, 2007.

[31] A. Krauss, C. Sternagel, R. Thiemann, C. Fuhs, and J. Giesl. Termination of Isabelle Functions via Termination of Rewriting. In *Proc. ITP*, LNCS 6898, pages 152–167, 2011.

[32] A. Bove, A. Krauss, and M. Sozeau. Partiality and Recursion in Interactive Theorem Provers—An Overview. *Mathematical Structures in Computer Science*, 26(1):38–88, 2016.

[33] A. Abel and B. Pientka. Wellfounded Recursion with Copatterns: A Unified Approach to Termination and Productivity. In *Proc. ACM ICFP*, pages 185–196, 2013.

[34] C. Hawblitzel, J. Howell, M. Kapritsos, J. R. Lorch, B. Parno, M. L. Roberts, S. Setty, and B. Zill. IronFleet: Proving Practical Distributed Systems Correct. In *Proc. ACM SOSP*, pages 1–17, 2015.

[35] J. R. Wilcox, D. Woos, P. Panchekha, Z. Tatlock, X. Wang, M. D. Ernst, and T. Anderson. Verdi: A Framework for Implementing and Formally Verifying Distributed Systems. In *Proc. ACM PLDI*, pages 357–368, 2015.

[36] H. Chen, D. Ziegler, T. Chajed, A. Chlipala, M. F. Kaashoek, and N. Zeldovich. Using Crash Hoare Logic for Certifying the FSCQ File System. In *Proc. ACM SOSP*, pages 18–37, 2015.

[37] R. Jung, D. Swasey, F. Sieczkowski, K. Svendsen, A. Turon, L. Birkedal, and D. Dreyer. Iris: Monoids and Invariants as an Orthogonal Basis for Concurrent Reasoning. In *Proc. ACM POPL*, pages 637–650, 2015.

[38] R. Kumar, M. O. Myreen, M. Norrish, and S. Owens. CakeML: A Verified Implementation of ML. In *Proc. ACM POPL*, pages 179–191, 2014.

[39] W. M. McKeeman. Differential Testing for Software. *Digital Technical Journal*, 10(1):100–107, 1998.

[40] X. Yang, Y. Chen, E. Eide, and J. Regehr. Finding and Understanding Bugs in C Compilers. In *Proc. ACM PLDI*, pages 283–294, 2011.

[41] V. Le, M. Afshari, and Z. Su. Compiler Validation via Equivalence Modulo Inputs. In *Proc. ACM PLDI*, pages 216–226, 2014.

[42] Z. Paraskevopoulou, C. Hriţcu, M. Dénès, L. Lampropoulos, and B. C. Pierce. Foundational Property-Based Testing. In *Proc. ITP*, LNCS 9236, pages 325–343, 2015.

[43] A. Q. Jiang, W. Li, S. Tworkowski, et al. Thor: Wielding Hammers to Integrate Language Models and Automated Theorem Provers. In *Proc. NeurIPS*, 2022.

[44] A. Q. Jiang, S. Welleck, J. P. Zhou, et al. Draft, Sketch, and Prove: Guiding Formal Theorem Provers with Informal Proofs. In *Proc. ICLR*, 2023.

[45] H. Wang, H. Xin, C. Zheng, et al. LEGO-Prover: Neural Theorem Proving with Growing Libraries. In *Proc. ICLR*, 2024.

[46] H. Xin, D. Guo, Z. Shao, et al. DeepSeek-Prover: Advancing Theorem Proving in LLMs through Large-Scale Synthetic Data. arXiv:2405.14333, 2024.

[47] A. Thakur, G. Tsoukalas, Y. Wen, J. Xin, and S. Chaudhuri. An In-Context Learning Agent for Formal Theorem-Proving. In *Proc. COLM*, 2024.